

Additional Material 2: The Art of Verification: Code Review and Evaluation in AI-Assisted Development

DOTÉ 3010 Course

Abstract

In the era of “Vibe Coding,” where Large Language Models (LLMs) accelerate code generation exponentially, the bottleneck of software engineering shifts from implementation to verification. This paper explores the critical role of human oversight in the AI-assisted development loop. We argue that while AI can write code, it lacks the contextual awareness to ensure long-term maintainability and factual correctness. We propose a dual-layer verification framework: strict automated testing (Unit/Integration) for structural integrity, and “LLM-as-a-Judge” evaluation for semantic validity. Furthermore, we reinstate the importance of peer code review—not as a critique of syntax, but as a safeguard against “hallucinated complexity” and security vulnerabilities. Drawing from best practices in modern software engineering, this document serves as a guide for students to transition from passive consumers of AI code to active architects of robust systems.

1 Introduction

The integration of Generative AI into the IDE has democratized software creation, allowing developers to generate boilerplate and complex logic with natural language prompts. However, this ease of generation introduces a new risk: the “Illusion of Competence.” Because the code looks correct and often runs without syntax errors, developers may skip the rigorous verification steps traditionally mandated in engineering.

Jeff Atwood, co-founder of Stack Overflow, famously posited that “peer code reviews are the single biggest thing you can do to improve your code” [1]. In the context of AI-generated code, this maxim holds even greater weight. AI models are probabilistic engines, not reasoning agents; they optimize for plausibility rather than correctness. Consequently, bugs in AI code are often subtle—logical gaps, security oversights (e.g., hardcoded secrets), or hallucinated dependencies—that automated linters miss.

This paper outlines a comprehensive strategy for verifying AI-generated software, focusing on three pillars: Automated Testing, AI-Assisted Debugging, and Semantic Evaluation.

2 Code Review: The Human Firewall

Despite the sophistication of LLMs, the human reviewer remains the ultimate arbiter of code quality. The goal of reviewing AI code is distinct from reviewing human code. Instead of focusing on syntax (which AI rarely gets wrong), the reviewer must focus on *intent* and *security*.

2.1 The “Just Do It” Philosophy

Adopting a mindset of continuous review is essential. Reviews catch approximately 60% of defects, compared to the 25% detection rate of unit testing alone. In a Vibe Coding workflow, where code volume is high, the review process prevents technical debt from accumulating into an unmanageable “Big Ball of Mud.”

2.2 Review Checklist for AI Code

When reviewing code generated by models like GPT-4 or Claude 3.5 Sonnet for the *Solver#42* project, students should employ the following checklist:

- **Secrets Management:** Ensure no API keys or database credentials are hardcoded. (e.g., verify `backend/core/config.py` uses environment variables).
- **Error Handling:** AI often assumes the “happy path.” Check if the code handles API failures (e.g., OpenAI timeouts) gracefully without crashing the server.
- **Logic Verification:** Does the generated prompt in `standard_answer_generator.py` actually align with the assignment instructions?

2.3 AI as the First Reviewer

Paradoxically, AI can also aid in the review process. Tools like Cursor allow developers to use the chat interface to “audit” files. A prompt such as “*Review this file for potential security risks and unused imports*” can act as a first pass, highlighting obvious issues for the human developer to investigate.

3 The Testing Pyramid in the AI Era

Reliable software requires a robust testing strategy. For AI-wrapper applications like *Solver#42*, the traditional testing pyramid is adapted to account for the cost and non-determinism of LLM calls.

3.1 Unit Testing with Mocking

Testing logic that depends on external APIs (like OpenAI) presents challenges: cost, latency, and randomness. Therefore, unit tests must employ **Mocking**. By using `unittest.mock`, developers can simulate the API’s response. This ensures that tests verify the *application logic* (e.g., prompt construction, response parsing) rather than the AI provider’s uptime.

```
1 @patch("backend.app.generator.client.chat.completions.create")
2 def test_prompt_construction(mock_create):
3     # Simulate AI response
4     mock_create.return_value.choices[0].message.content = "Mocked Answer"
5
6     # Run function
7     result = generate_answer("Math 101", "Solve X")
8
```

```

9      # Verify the PROMPT sent to AI, not just the result
10     call_args = mock_create.call_args
11     assert "Math 101" in call_args[1]['messages'][1]['content']

```

Listing 1: Example of Mocking OpenAI in Tests

3.2 Integration Testing

Integration tests verify the flow between components (API → Database → Background Task). In *Solver#42*, we utilize `sqlite:///memory:` with `StaticPool` to create ephemeral, fast test databases that mimic the production PostgreSQL environment without the overhead.

4 Semantic Evaluation: LLM-as-a-Judge

Passing unit tests does not guarantee that the AI’s output is useful. The final frontier of verification is **Evaluation** (Eval). Since there is often no single “correct” answer for generative tasks, we employ an approach known as “LLM-as-a-Judge,” detailed in recent literature [3].

4.1 The Evaluation Workflow

1. **Golden Set:** Curate a small dataset of diverse inputs (e.g., a calculus problem, a history essay prompt).
2. **Generation:** Run the application to produce answers.
3. **Adjudication:** Use a strong model (like GPT-4o) to grade the generated answer against a rubric or a reference answer.

4.2 Web Search Verification

For factual queries, we can enhance evaluation by equipping the “Judge” with web search tools. As demonstrated in the course notebook, an Evaluation Agent can: 1. Read the generated output (e.g., “The capital of Australia is Sydney”). 2. Perform a web search to verify the claim. 3. Flag the hallucination (Correct answer: Canberra) and penalize the score.

5 Conclusion

As software engineering evolves, the ability to write code becomes secondary to the ability to *verify* code. “Vibe Coding” is not an excuse for sloppy engineering; rather, it demands a higher standard of rigour in testing and review. By combining the deterministic safety of automated tests with the semantic understanding of AI-based evaluation, developers can build systems that are not only fast to create but reliable to run.

References

- [1] Atwood, J. (2006). *Code Reviews: Just Do It*. Coding Horror Blog. Retrieved from <https://blog.codinghorror.com/code-reviews-just-do-it/>

- [2] GitHub. (2023). *How to review code effectively: A GitHub staff engineer’s philosophy*. GitHub Blog.
- [3] Zheng, L., et al. (2024). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. arXiv preprint arXiv:2306.05685.